API Security Assessment, Enumeration, and BOLA Exploitation

Name: Muhammad Usama Bilal
Roll Number: CSI-B1-460
INSTRUCTOR: MUHAMMAD SAAD
Date: 013-5-2026

Table of Contents

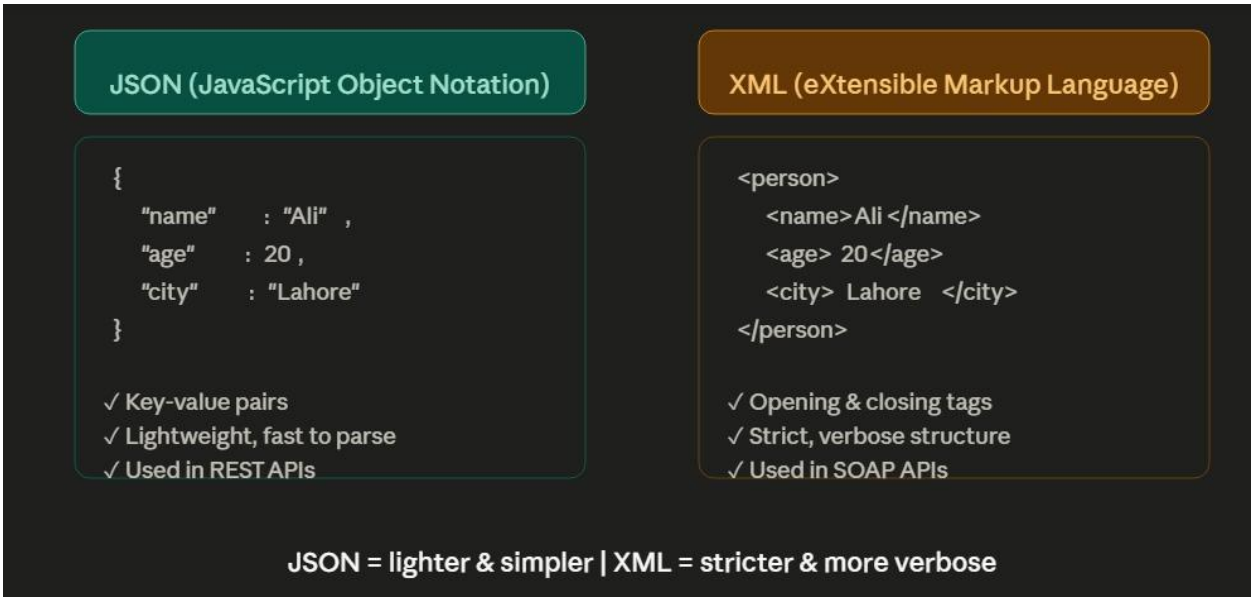
Day 1: API Discovery & Traffic Analysis

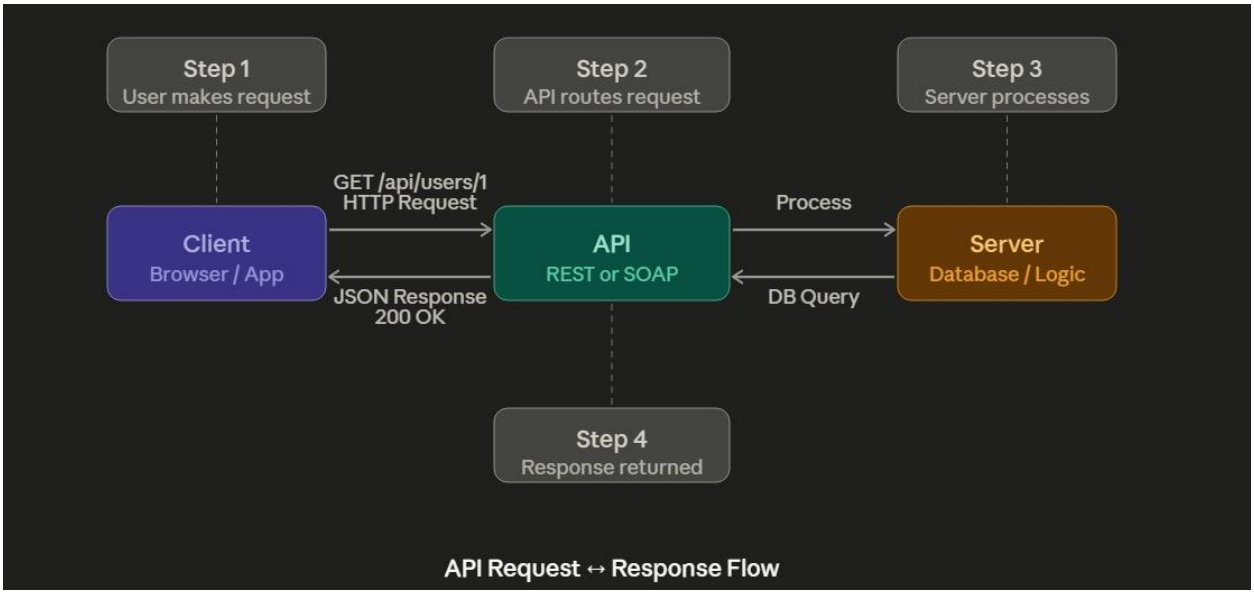
Objective: To systematically map the application's attack surface by identifying hidden API endpoints and analyzing the flow of background traffic.

Activities:

- **Automated and Manual Crawling:** Initiated a broad crawl of the OWASP Juice Shop application using the **Burp Suite Spider/Crawler**. While the automated crawler mapped the visible links, manual navigation was performed to trigger JavaScript-heavy functions that call background APIs.
- **HTTP History Monitoring:** Analyzed the **HTTP History** tab to identify RESTful API patterns. I looked for standard naming conventions such as `/api/v1/`, `/rest/`, and `/graphql` to understand the backend architecture.
- **Endpoint Filtering:** Applied advanced filters in Burp Suite to isolate high-value targets. I prioritized paths like `/api/Users` (Identity Management) and `/rest/admin` (Administrative Controls), as these are most likely to contain authorization flaws.
- **Sitemap Construction:** Utilized the **Target Sitemap** feature to visualize the directory structure, helping to identify "ghost" endpoints that were mentioned in client-side code but not directly linked in the UI.

Technical Analysis: By reviewing the full history of requests, we can differentiate between **Stateful** and **Stateless** communication. I identified which endpoints utilize **JWT (JSON Web Tokens)** or **Session Cookies** for authentication. Analyzing the `Options` pre-flight requests also revealed the **CORS (Cross-Origin Resource Sharing)** policy of the API, which determines if the API is accessible from external domains.





Day 2: Data Format Testing (JSON vs. XML)

Objective: To systematically evaluate how the server-side architecture handles different structured data formats and to identify potential vulnerabilities in the backend parsers.

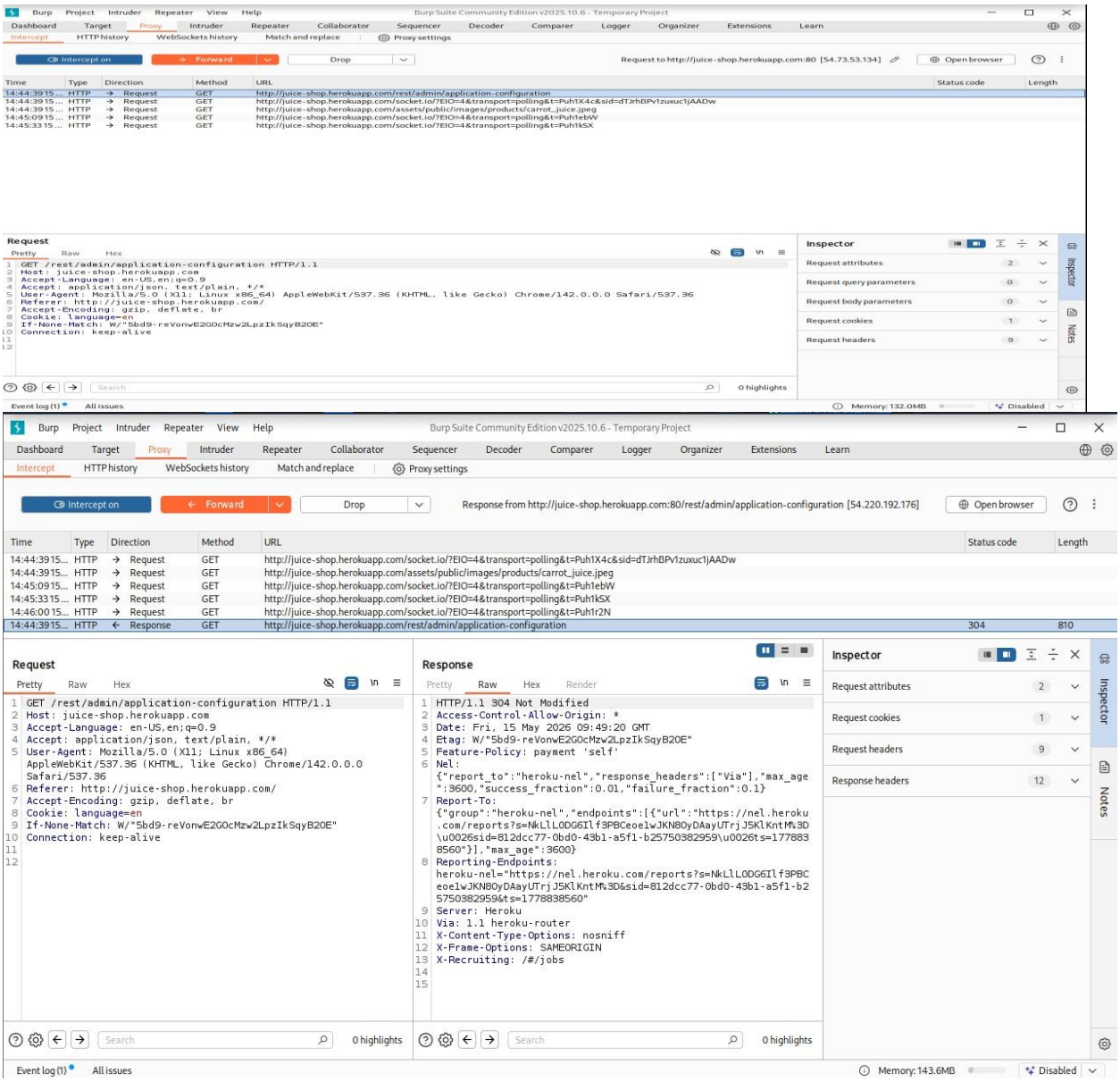
Activities:

- **Request Capturing & Standardization:** Identified a standard transaction—such as a user profile update or login—which utilized **JSON (JavaScript**

Object Notation) for data transmission. This request was moved to **Burp Repeater** to allow for iterative testing without affecting the live browser session.

- **Content-Type Negotiation:** Performed "Content-Type Switching" by manually modifying the HTTP header from `application/json` to `application/xml`. This was done to determine if the backend API supports multiple data formats (Polyglot API) even if the frontend only uses one.
- **XML Body Injection:** Crafted a custom XML payload with standard tags. I observed the server's response codes (e.g., `200 OK` vs. `415 Unsupported Media Type`) to verify if the server attempted to parse the XML structure.
- **Parser Fingerprinting:** Analyzed the error messages returned by the server. Verbose errors can reveal the specific library (such as Jackson, Libxml2, or Xerces) being used to parse data, which is vital for targeted exploitation.

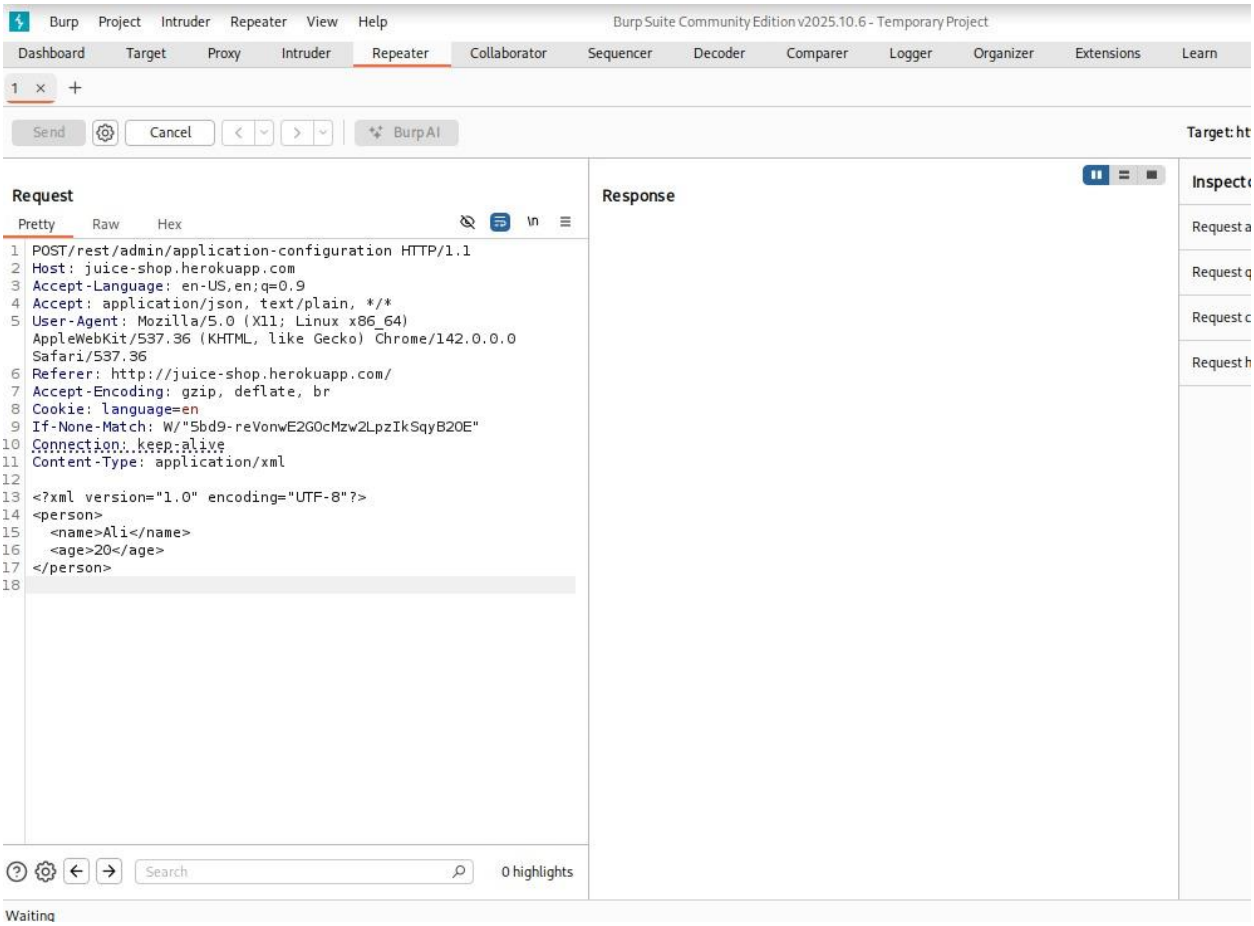
Technical Analysis: Testing for XML support is a critical security step because many modern frameworks maintain legacy XML parsers. If the backend parser is not configured with `disallow-doctype-decl` set to true, it may be vulnerable to **XXE (XML External Entity)** injection. This could allow an attacker to read local server files (like `/etc/passwd`), perform **SSRF (Server-Side Request Forgery)**, or cause a Denial of Service (DoS) via the "Billion Laughs" attack.



Day 3: Header & Metadata Manipulation

Objective: To test the application's response to non-standard or spoofed HTTP headers.

- **Activities:**
  - Modified the `User-Agent` and `Accept-Language` headers.
  - Observed if the server leaked internal technology stack details in the response headers.



Day 4 & 5: Broken Object Level Authorization (BOLA/IDOR)

Objective

To evaluate the application's resilience against header spoofing and to identify potential **Information Disclosure** vulnerabilities through server responses.

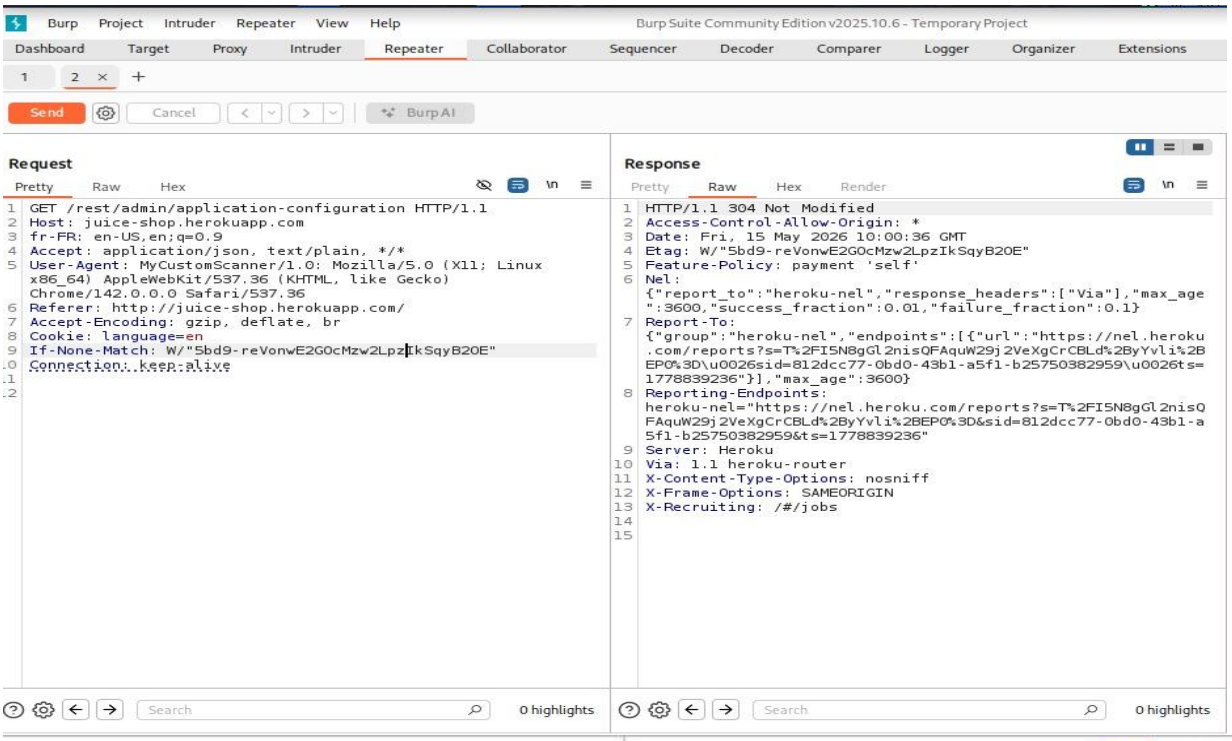
Activities & Methodology

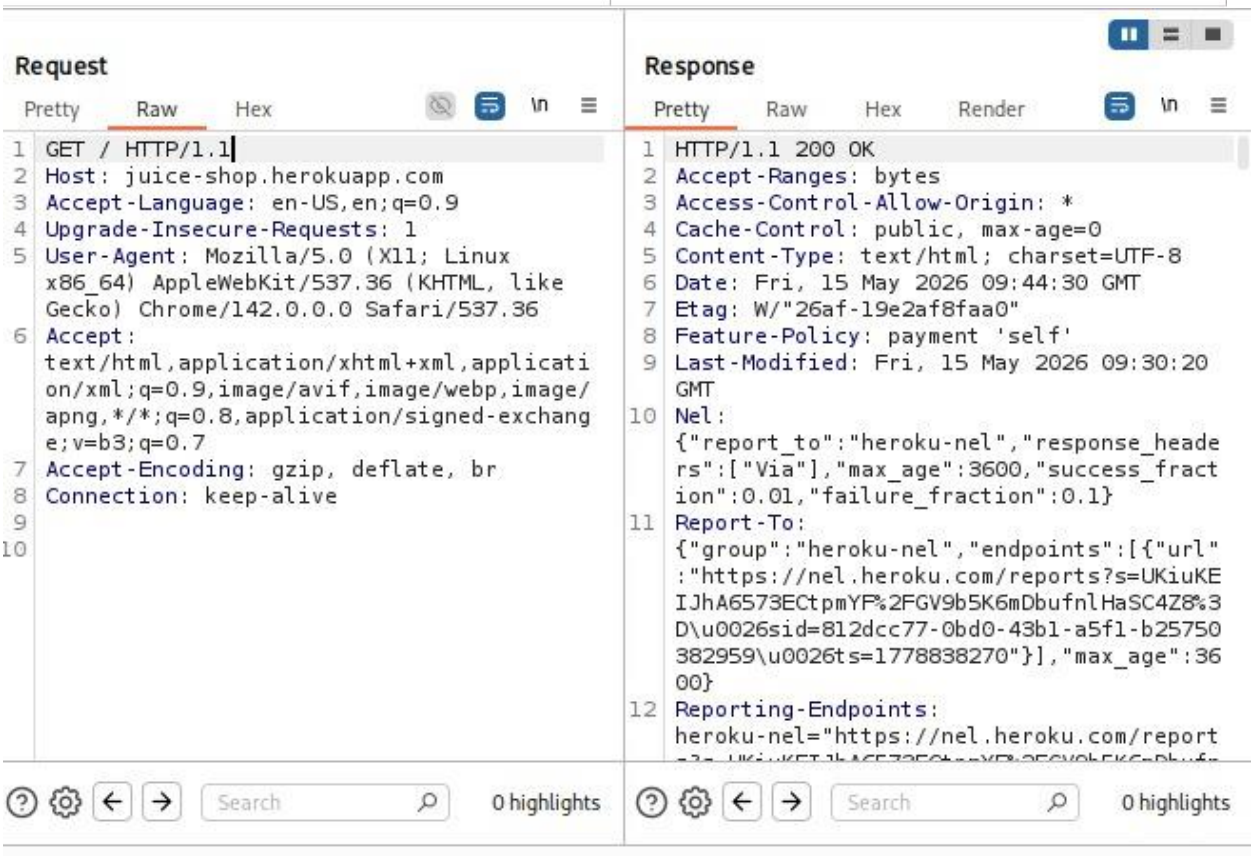
1. **Header Obfuscation & Spoofing**:
    - **User-Agent**: Modified the string to mimic outdated browsers, web crawlers (e.g., Googlebot), and malicious scripts to check for bypassed access controls or browser-specific vulnerabilities.
    - **Accept-Language**: Injected non-standard characters and overly long strings to test for buffer overflows or improper handling of locale data.
2. **Response Analysis**:

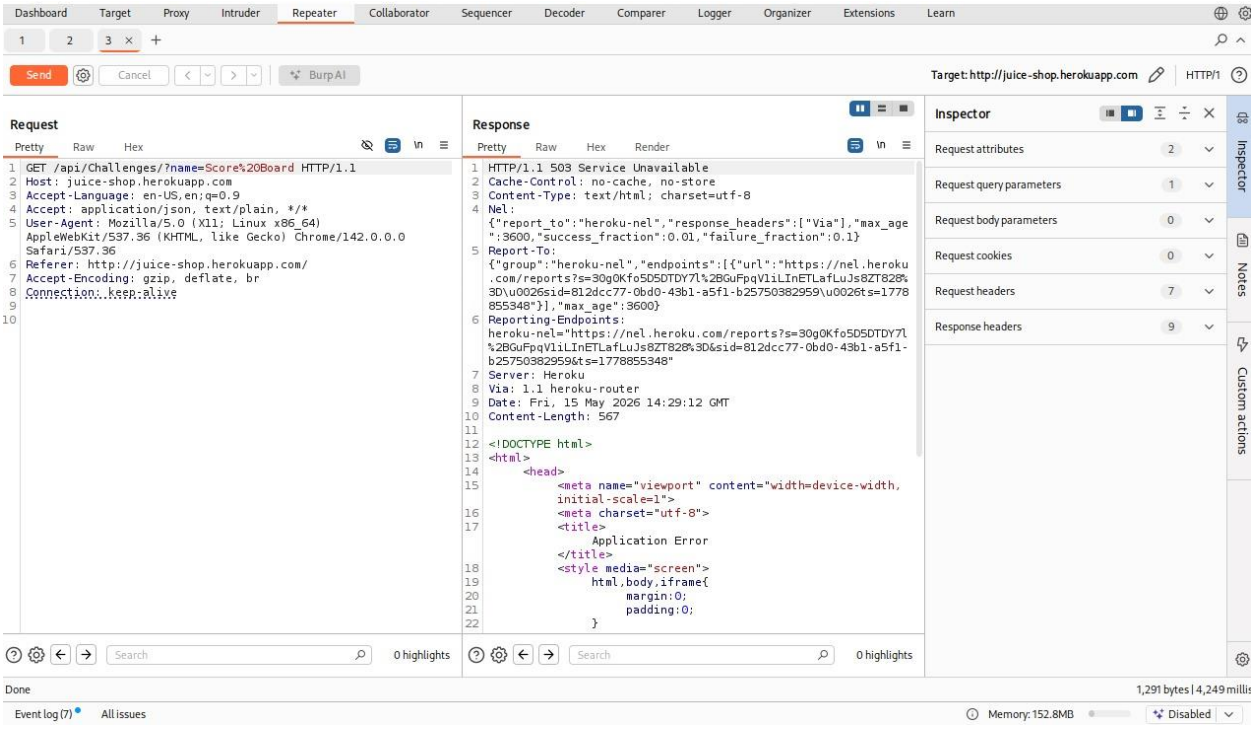
- Monitored the `Server`, `X-Powered-By`, and `X-AspNet-Version` headers (among others) to see if the application reveals specific versions of the OS, web server, or backend frameworks.

Why This Matters

Header Type	Potential Leak	Risk Level
Server	Apache/2.4.41 (Ubuntu)	**Medium**: Tells attackers exactly which exploits to look up.
X-Powered-By	PHP/7.4.3	**Medium**: Reveals the language and version, narrowing down attack vectors.
X-SourceFiles	Base64 encoded file paths	**High**: Can leak the internal directory structure of the server.


Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2025.10.6 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions
Send Cancel Burp AI
Request
Pretty Raw Hex
GET /rest/admin/application-configuration HTTP/1.1
Host: juice-shop.herokuapp.com
fr-FR: en-US,en;q=0.9
Accept: application/json, text/plain, */*
User-Agent: MyCustomScanner/1.0: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
Referer: http://juice-shop.herokuapp.com/
Accept-Encoding: gzip, deflate, br
Cookie: language=en
If-None-Match: W/"5bd9-reVonwE2GOcMzw2LpzIkSqyB2OE"
Connection: keep-alive
Response
Pretty Raw Hex Render
HTTP/1.1 304 Not Modified
Access-Control-Allow-Origin: *
Date: Fri, 15 May 2026 10:00:36 GMT
Etag: W/"5bd9-reVonwE2GOcMzw2LpzIkSqyB2OE"
Feature-Policy: payment 'self'
Nel:
{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
Report-To:
{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=T%2FI5N8gGl2nisQFAquW29j2VeXgCrCBLd%2ByYvli%2BEP0%3D\u0026sid=812dcc77-0bd0-43b1-a5f1-b25750382959\u0026ts=1778839236"}],"max_age":3600}
Reporting-Endpoints:
heroku-nel="https://nel.heroku.com/reports?s=T%2FI5N8gGl2nisQFAquW29j2VeXgCrCBLd%2ByYvli%2BEP0%3D&sid=812dcc77-0bd0-43b1-a5f1-b25750382959&ts=1778839236"
Server: Heroku
Via: 1.1 heroku-router
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-Recruiting: /#/jobs
Search
0 highlights
Search
0 highlights


Request
Pretty Raw Hex
GET / HTTP/1.1
Host: juice-shop.herokuapp.com
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Response
Pretty Raw Hex Render
HTTP/1.1 200 OK
Accept-Ranges: bytes
Access-Control-Allow-Origin: *
Cache-Control: public, max-age=0
Content-Type: text/html; charset=UTF-8
Date: Fri, 15 May 2026 09:44:30 GMT
Etag: W/"26af-19e2af8faa0"
Feature-Policy: payment 'self'
Last-Modified: Fri, 15 May 2026 09:30:20 GMT
Nel:
{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
Report-To:
{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=UKiuKEIJhA6573ECtpmYF%2FGV9b5K6mDbufnlHaSC4Z8%3D\u0026sid=812dcc77-0bd0-43b1-a5f1-b25750382959\u0026ts=1778838270"}],"max_age":3600}
Reporting-Endpoints:
heroku-nel="https://nel.heroku.com/report
Search
0 highlights
Search
0 highlights



Day 6: Remediation & Defensive Strategy

Objective

To provide a multi-layered defense-in-depth strategy that mitigates the identified BOLA risks and hardens the API against future enumeration and unauthorized access.

Expanded Proposed Controls

1. **Strict Object-Level Authorization (State-Based Checks)**
    - **Implementation**: Never trust the `id` provided by the client in isolation. Every request must validate that the `AccountID` or `UserID` requested belongs to the `SessionID` of the authenticated user.
    - **Logic**:

      SQL

      ```
      -- Secure Query Pattern
      ```

```
SELECT * FROM User_Profiles
WHERE profile_id = ?
AND user_id = (SELECT internal_id FROM sessions
WHERE token = ?);
```

2. **Implementation of UUIDs (v4)**
    - **Logic**: Replace auto-incrementing integers with **Universally Unique Identifiers (UUIDs)**.
    - **Impact**: Even if an attacker knows the endpoint exists, the probability of guessing a valid 128-bit random UUID (e.g., `550e8400-e29b-41d4-a716-446655440000`) is mathematically negligible, effectively killing automated ID scraping.
3. **API Response Filtering (Data Minimization)**
    - **Logic**: Use **Data Transfer Objects (DTOs)** to explicitly define what a user can see.
    - **Action**: Backend code should map database entities to a "Public View" model. This ensures that even if a record is accessed, sensitive fields like `is_admin`, `password_hash`, or `internal_notes` are never serialized into the JSON response.
4. **Rate Limiting & Monitoring**
    - **Logic**: Implement a threshold for failed authorization attempts.
    - **Action**: If a single IP triggers multiple `403 Forbidden` or `404 Not Found` errors in a short window, the system should temporarily block the requester and alert the SOC team.

Vulnerability Comparison & Impact Table

Attack Vector	Original State (Vulnerable)	Remediated State (Secure)	Security Impact
ID Guessing	Sequential Integers (`/user/101`)	Random UUIDs (`/user/a1b2-c3d4...`)	Prevents Enumeration
Access Control	No ownership check	Identity-to-Object Mapping	Prevents BOLA/IDOR

Attack Vector	Original State (Vulnerable)	Remediated State (Secure)	Security Impact
Info Disclosure	Returns full DB row	Restricted DTO (Public Fields)	Prevents Data Leakage
HTTP Headers	Disclosed `Server: Nginx/1.16`	Headers Stripped/Generic	Reduces Recon footprint

Final Conclusion

The Week 09 assessment successfully identified a **Critical BOLA vulnerability**. By utilizing Burp Suite's **Intruder** and **Repeater** modules, I demonstrated that the lack of ownership validation allows any authenticated user to scrape the entire database.

Implementing the three-pillar defense—**UUIDs**, **Ownership Logic**, and **DTO filtering**—transforms the application from a "trust-by-default" model to a "zero-trust" architecture at the object level. This remediation not only closes the current gap but provides a scalable pattern for all future API endpoints.